



Java Inheritance

Section 1: Learn

What is Inheritance?

Inheritance is a feature in Java where one class (child) **inherits properties and methods** from another class (parent).

- Promotes **code reusability**
 - Helps in **method overriding**
 - Implements **"is-a"** relationship
-

Why Do We Need Inheritance?

- Reduces code duplication
 - Makes maintenance easier
 - Allows enhancements without changing existing code
 - Follows the **DRY** (Don't Repeat Yourself) principle
-

How to Use Inheritance

Use the **extends** keyword:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes sound");  
    }  
}  
  
class Dog extends Animal {
```



```
void bark() {  
    System.out.println("Dog barks");  
}  
}
```

Now **Dog** has both **sound()** and **bark()** methods.

Types of Inheritance in Java

1. Single Inheritance

One class inherits from another:

```
class Parent { }  
class Child extends Parent { }
```

2. Multilevel Inheritance

A class inherits from a class that also inherited from another class.

```
class A { }  
class B extends A { }  
class C extends B { }
```

3. Hierarchical Inheritance

Multiple classes inherit from one parent.

```
class Vehicle { }  
class Car extends Vehicle { }  
class Bike extends Vehicle { }
```



4. Hybrid Inheritance

Java does **not support hybrid inheritance directly** using classes (to avoid ambiguity). But it can be achieved using **interfaces**.

Keywords Used in Inheritance

super

Used to refer to **parent class methods and variables**.

```
class Animal {  
    void sound() { System.out.println("Animal sound"); }  
}  
  
class Cat extends Animal {  
    void sound() {  
        super.sound(); // calls parent sound  
        System.out.println("Meow");  
    }  
}
```

this

Refers to the **current class object**.

```
class Box {  
    int length;  
    Box(int length) {  
        this.length = length;  
    }  
}
```



```
}
```

this() and **super()**

- **this()**: Calls **constructor** of the same class.
- **super()**: Calls **constructor** of the parent class.

```
class A {  
    A() { System.out.println("Parent"); }  
}
```

```
class B extends A {  
    B() {  
        super(); // calls A's constructor  
        System.out.println("Child");  
    }  
}
```

extends

Used to inherit a class.

implements

Used when a class implements an **interface**.

```
interface Printable {  
    void print();  
}
```

```
class Report implements Printable {  
    public void print() {
```



```
        System.out.println("Report printed");
    }
}
```

Real World Example

```
class Employee {
    String name = "Rahul";
    void work() {
        System.out.println(name + " is working");
    }
}

class Manager extends Employee {
    void manage() {
        System.out.println(name + " is managing team");
    }
}

public class Company {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.work(); // inherited
        m.manage(); // own method
    }
}
```



Section 2: Practice

Try Single Inheritance

```
class Fruit {  
    void taste() { System.out.println("Sweet"); }  
}  
  
class Mango extends Fruit {  
    void color() { System.out.println("Yellow"); }  
}
```

Try Multilevel Inheritance

```
class Grandparent {  
    void wisdom() { System.out.println("Old and wise"); }  
}  
  
class Parent extends Grandparent {  
    void teach() { System.out.println("Teaches values"); }  
}  
  
class Child extends Parent {  
    void play() { System.out.println("Plays games"); }  
}
```



Section 3: Know More (FAQs)

Can I inherit from multiple classes?

No. Java does not support **multiple inheritance** with classes, but allows it via **interfaces**.

Why doesn't Java allow multiple inheritance with classes?

To avoid **ambiguity** or the **diamond problem**.

What is the default superclass of all classes?

Object class is the root class of all Java classes.

When should I use **super?**

When you want to:

- Call parent class method
- Access parent class constructor or variables

Can constructors be inherited?

No, but the child class can call the parent's constructor using **super()**.